

CRAFT — Pitch Package

Audited retrieval-augmented AI assistant for CSA mission engineering and finance.

1. Design & Architecture (L2)

L2 · Containers · Actors · External Systems

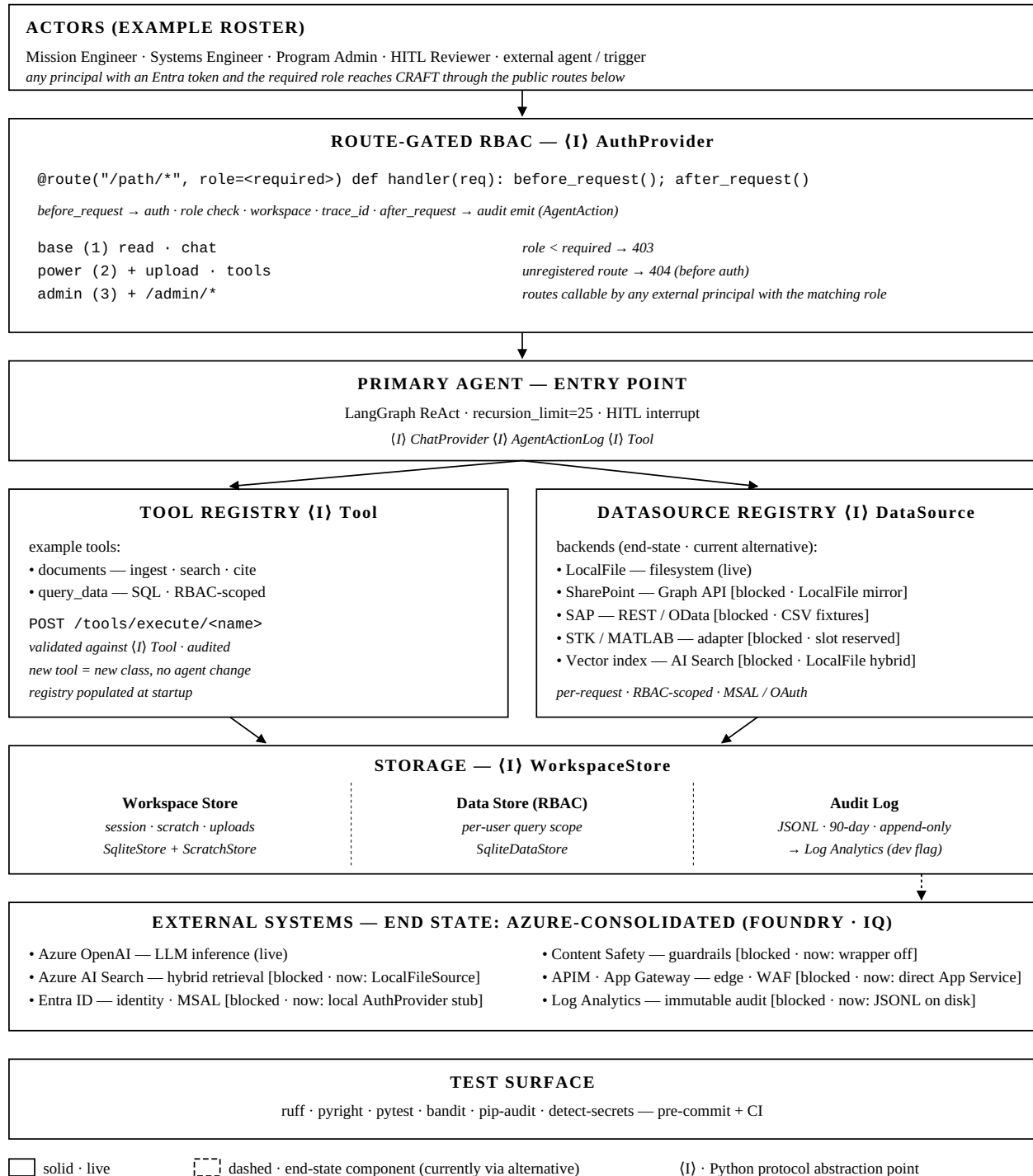


Figure 1 · Actors, routes, containers, data, external systems, and the audit perimeter.

The system is a Flask + Dash application fronted by a route-gated RBAC layer. The design decision driving everything else: **every entry point is a REST route, every route is role-checked, and every step emits an audit record**. That shape lets the Primary Agent, the Tool Registry, and the DataSource Registry stay loosely coupled behind Python protocols — the agent calls /tools/execute/<name> and

never knows which concrete tool runs; a tool queries through `(I) DataSource` and never knows whether it's reading a local file or SharePoint. The audit log sits across all three so trace reconstruction stays complete regardless of which backend serves a given call.

Choosing Flask over a heavier framework was deliberate: it keeps the public surface a plain HTTP API that any external agent, scheduler, or CI runner can hit directly, so the whole product is deployable anywhere a container runs and callable from anywhere a trigger lives — a Teams bot, a cron job, an upstream orchestrator agent. Dash is used only for the HITL reviewer UI, which lives inside the same auth perimeter and talks to the same routes.

L3 · Internal Clusters

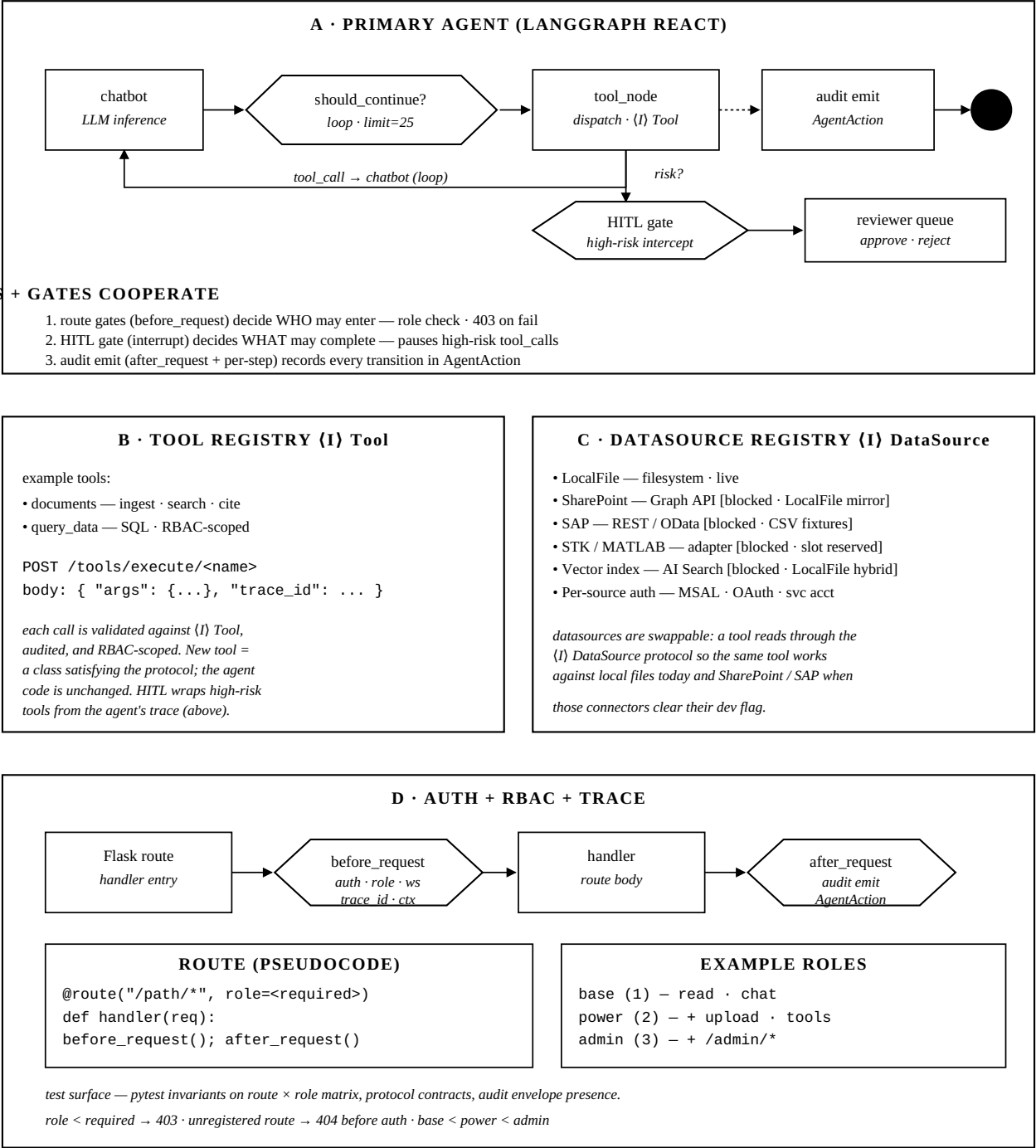


Figure 2 · Agent loop with HITL branch, tool registry, data-source registry, and auth cluster.

The Primary Agent is a LangGraph ReAct graph with `recursion_limit = 25` and a HITL interrupt branch off the `tool_node`. Routes gate who may enter; the HITL gate decides what may complete. Both are first-class graph edges, not middleware tricks, so the invariants are reachable from `pytest`. The Tool and DataSource registries are not inheritance hierarchies — they are Python protocols registered at startup; a new tool is a class that satisfies `(I) Tool`, dropped in a module, and the registry picks it up with no changes to the agent.

3. Use Cases

Use-Case Lanes · E2 · F1 · F4

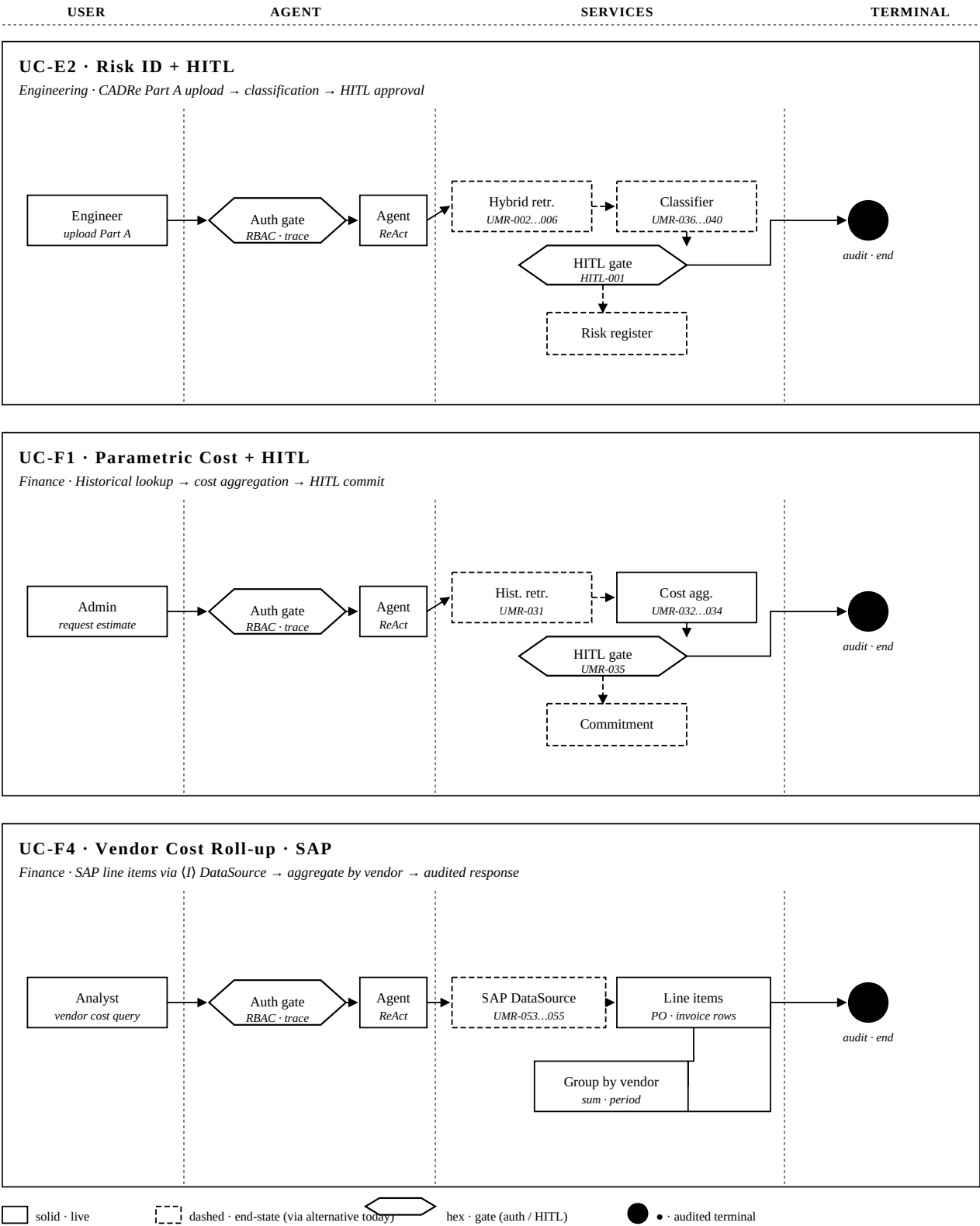


Figure 3 · Three use-case lanes left-to-right with UC IDs.

Three lanes exercise the full spine: **UC-E2** (Engineering risk ID + HITL) runs the full retrieval → classify → human-gate → register path. **UC-F1** (parametric cost + HITL) runs the historical-lookup → aggregate → human-gate → commit path. **UC-F4** (vendor cost via SAP) runs the connector-based retrieval → aggregate path, with the SAP backend behind the same **{I}** `DataSource` as `LocalFile` so the lane shape is identical whichever backend is live. Every other mission flow (budget scenarios, anomaly triage, bilingual reports, cross-mission compare) attaches as a new `Tool` + optional `DataSource` along the same lanes.

4. Design Decisions & Component Rationale

Component	Role	Why this choice
Flask	Public HTTP surface	Mature, standards-based WSGI; routes are the integration contract — any external agent or scheduler can trigger CRAFT with no SDK.
LangGraph	Primary agent runtime	First-class <code>interrupt</code> for HITL, bounded recursion, graph-level test anchors. Actively maintained by LangChain Inc.; standard ReAct pattern.
Dash	HITL reviewer UI	Shares the Flask process and auth, so the reviewer surface inherits RBAC without a second deployment.
Azure OpenAI	LLM inference	Canadian-region deployment satisfies PBMM data-residency; enterprise SLA; swappable behind {I} <code>ChatProvider</code> .
Azure AI Search	Hybrid retrieval	Vector + BM25 in one service; managed; swappable behind {I} <code>DataSource</code> .
Azure Content Safety	Guardrails	Microsoft-maintained moderation; invoked through a tool wrapper so the control plane stays swappable.
Azure Container Apps	Code-exec sandbox	Isolated per-request containers; standard scale-to-zero; satisfies UMR-093.
Azure App Service (PBMM)	Runtime	PBMM Canadian region, managed TLS, blue-green deploy, zero-trust networking into APIM.
APIM + App Gateway	Edge	WAF + rate limit at the perimeter; terminates mTLS; makes CRAFT addressable without exposing App Service directly.
Entra ID + MSAL	Identity	CSA-standard identity; MSAL is Microsoft-maintained; token flow identical for human and agent callers.
Log Analytics	Audit sink	Immutable retention policy satisfies UMR-027 audit grade; KQL-queryable.
Postgres	HITL approval queue	Durable queue state across restarts; mature; backs the Dash reviewer UI.
SQLite	WorkspaceStore / DataStore	File-local, zero-ops for scratch + per-user scoped data; swappable behind {I} <code>WorkspaceStore</code> when a managed DB is in place.
Python protocols	Extension boundary	Structural typing — a new tool or data source is a class satisfying the protocol, no base class, no registry edit, no agent change.
pytest + pyright + ruff	Architecture invariants	Industry-standard, actively maintained; invariants ship as tests (route × role matrix, protocol contracts, audit envelope present).

5. Abstraction for the Future

Four protocol boundaries carry the extensibility story. Each one is a single Python file; adding a backend behind one of them is additive and leaves the agent untouched.

Protocol	What it abstracts	How future growth lands
{I} <code>Tool</code>	Any callable capability the agent can invoke	New capability = new class satisfying the protocol. Agent code unchanged. Validated by <code>test_tool_contract</code> .
{I} <code>DataSource</code>	Any corpus or system of record	SharePoint, SAP, STK, a future vector index — each is a class; the agent calls them identically.
{I} <code>ChatProvider</code>	The LLM runtime	Lets us swap inference backends behind one interface; sub-agents become new graph nodes under the same provider.
{I} <code>WorkspaceStore</code> / {I} <code>AgentActionLog</code>	Session state and audit sink	SQLite today, managed DB tomorrow; JSONL today, Log Analytics tomorrow — same call site.

External accessibility. Routes are the integration contract. `POST /chat/send`, `POST /chat/upload`, `POST /tools/execute/<name>` and `GET /admin/*` are reachable by any external principal that holds an Entra token with the required role — a human in the Dash UI, a scheduled job, a Teams bot, or an upstream orchestrator agent. That means CRAFT can be *triggered* (cron, webhook, agent) and *embedded*

(as a tool for a larger agent) without new code. Deployment is a container image; the APIM policy decides which callers can reach which routes.

6. Requirements → Design

CAPABILITY	REQUIREMENTS	ABSTRACTION POINT	USE CASES
Primary agent	UMR-001/011/013/016/018/019/094/095, AAR-001/003/006, ASG-005	LangGraph ReAct · {I} ChatProvider · recursion_limit=25	E2, F1, F4
Retrieval + RAG	UMR-002/003/004/005/006/009/010/060/067, ARR-002-007	Azure AI Search (hybrid vector + BM25) · {I} DataSource + {I} Tool	E2, F1
Audit + traceability	UMR-015/027/045/061/062/093, AAR-005, ASG-003	AgentAction envelope · {I} AgentActionLog · JSONL → Log Analytics	E2, F1, F4
HITL approvals	UMR-021-026/035/039/044/049, HITL-001-007, ASG-002	LangGraph interrupt · Postgres queue · Dash review UI	E2, F1
RBAC + data scoping	UMR-012/017/058/065/066, AAR-002/007	_ROUTE_RULES + _ROLE_LEVEL · SqliteDataStore · QueryTool.set_user	E2, F1, F4
Session + workspace	UMR-014/092/095	{I} WorkspaceStore → SqliteStore + SessionScratchStore	all
Guardrails + containment	UMR-020/029/030, ASG-001/004/011	Azure Content Safety · APIM rate limit · Container Apps sandbox	all
Connectors + external	UMR-051-055	{I} DataSource + MSAL · Flask REST · SAP / STK / MATLAB adapters	F4
Domain capabilities	UMR-007/008/031-034/036-038/040-043/046-048/050/075, ARR-001	New {I} Tool instances over existing abstraction points	E2, F1
Deployment, SLA	UMR-028/056/057/059/064/069/070/074/085, ASG-006	Azure App Service PBMM · APIM + WAF · SSC LaunchPad HA · git config	all

7. Blockages — End State vs Current Alternative

The architecture is designed for its end state: an Azure-consolidated runtime (Foundry, AI Search, Content Safety, Container Apps, Log Analytics, Entra ID, App Service, APIM) with CSA-owned data staying in Azure Canadian regions. Consolidating on one cloud shortens the vendor surface, puts every dependency under one procurement path, and lets us reuse the same identity, networking, and audit plane across every component. **Each row below names an end-state component, the owner it is waiting on, and the current alternative CRAFT runs against while that component is in the pipeline.** The alternative is wired through the same {I} abstraction point as the end-state component, so moving to the end state is a configuration change, not a rewrite.

Models are the one deliberate exception: inference lives in Azure OpenAI today and may route through Azure AI Foundry (Azure IQ) tomorrow, but the {I} ChatProvider abstraction keeps the agent indifferent to where inference actually runs. Some data (session scratch, local corpora) stays on-box by design even in the end state.

END-STATE COMPONENT	ROLE	OWNER	CURRENT ALTERNATIVE IN THE RUNNING SYSTEM
Azure AI Search (serverless)	Hybrid retrieval + citations behind {I} DataSource	IT	LocalFileSource — filesystem search; same {I} DataSource protocol, no agent change on cutover.
Azure Log Analytics + immutability policy	Tamper-proof audit sink behind {I} AgentActionLog	IT	JSONL append-only files on local disk, 90-day retention; same envelope, same sink interface.
Azure Content Safety	Moderation hook on every tool invocation	Procurement	Guardrails off at the tool wrapper; call site exists, moderation call is a config flag.
Azure Container Apps sandbox	Isolated code execution for CodeExecTool	Procurement	CodeExecTool disabled; the abstraction is present so routing switches at deploy time.
Entra ID (app registration)	Identity + MSAL token flow for routes	IT	Local auth provider stub satisfying {I} AuthProvider ; token issuance mocked for tests.
SharePoint connector	CSA corpus behind {I} DataSource	IT + CSA	LocalFileSource reads a mirror of the corpus; identical call site.
SAP connector	Vendor cost data behind {I} DataSource	Finance + Procurement	CSV fixtures loaded through a LocalFileSource adapter; UC-F4 lane runs end-to-end.
STK / MATLAB adapters	Mission simulation / analysis {I} Tool	Procurement (licenses)	Not instantiated; {I} Tool registration slot reserved.
Azure AI Foundry / Azure IQ (model routing)	Routes {I} ChatProvider across model families	IT	Azure OpenAI direct (Canadian region); same {I} ChatProvider , swap at config.
SSC LaunchPad HA	HA topology on PBMM landing zone	SSC	Single-region App Service deployment; infra-only change at cutover.
CSA risk taxonomy	Risk tier definitions for the classifier + HITL	CSA Domain	Stub taxonomy drives the classifier under test; swap by schema import.
Historical mission DB	Parametric cost reference set	CSA Finance	Fixture CSV through LocalFileSource ; UC-F1 lane runs on fixtures.

8. System-Health Tooling

Health tooling runs on every commit (pre-commit) and in CI. These are not blockages — they are the invariants that keep the architecture honest while the end-state components come online.

TOOL	ROLE	WHERE IT RUNS
ruff	Lint — flake8 + isort + pyupgrade rules	pre-commit + CI
pyright (strict)	Type checking — protocol conformance for {I} Tool , {I} DataSource	pre-commit + CI
pytest + coverage	Architecture invariants, route × role matrix, protocol contracts, audit-envelope presence (156 tests, 91 % coverage)	pre-commit + CI
bandit	Security scan on source	pre-commit
pip-audit	Dependency CVE audit	pre-commit
detect-secrets	No credentials in git	pre-commit